

Smart Electricity Bill Analyzer — Product Requirements Document (PRD)

Field	Detail
Product	Smart Electricity Bill Analyzer
Type	Mini project (web application)
Version	1.0 (Draft)
Tech Stack	HTML, CSS, JavaScript (frontend) · Python + Flask (backend) · MongoDB (database) · Chart.js (charts)
Source	Derived from the two project infographics (Flow Chart + Mini Project Blueprint)

1. Overview

1.1 Product Summary

Smart Electricity Bill Analyzer is a web application that helps households understand, track, predict and reduce their electricity cost. Users can either **enter meter readings manually** or **upload an existing bill (PDF/image)**. The system calculates units consumed, applies slab-wise tariff, fixed charges, duty/taxes and rebates, stores monthly history, shows charts, predicts next month's bill, gives energy-saving tips, and lets the user download a PDF report.

1.2 Problem Statement

Consumers receive a monthly bill but rarely understand *why* it is high, how it is calculated, or which appliances drive the cost. There is no easy place to see history, trends or next month's estimate.

Core problem: Understand and reduce electricity cost.

1.3 Goals / Objectives

1. Calculate the electricity bill accurately using slab-wise tariff.
2. Track monthly usage and keep a complete bill history.
3. Identify high-consumption appliances.
4. Show usage and cost through interactive charts.
5. Predict the next month's bill.
6. Give personalised saving tips.
7. Provide a downloadable PDF analysis report.

1.4 Non-Goals (v1)

- Real-time smart-meter/IoT integration.
- Online bill payment.
- Native mobile app.
- Multi-utility support (water, gas).

1.5 Success Metrics

Metric	Target
Bill calculation matches an actual bill	within $\pm 2\%$
Bill data extraction accuracy from uploads (typed PDFs)	$\geq 85\%$ of key fields
Prediction error (MAPE) with ≥ 6 months of data	$\leq 15\%$
Dashboard load time	< 3 s
Core flow (register $\rightarrow$ add bill $\rightarrow$ see dashboard)	< 3 min

2. Target Users & Personas

Persona	Description	Need
Household Consumer (primary)	Homeowner/tenant paying a monthly bill	Understand bill, save money
Student / Hostel / PG resident	Shared or budget-conscious usage	Estimate bill, split cost
Admin (optional, v1.1)	Maintains tariff data	Update slabs, view usage of app

3. Scope — Modules

The 8 modules from the flow chart, mapped to the 5 main modules from the blueprint (user input, bill calculator, database, dashboard, recommendations).

#	Module	Purpose	Output / Benefit
1	User Registration & Login	Create account and log in securely	Secure access
2	Upload / Enter Bill	Upload PDF/image or enter meter readings manually	Bill data extracted and stored
3	Monthly Bill History	View and manage all previous bills	Complete history in one place
4	Consumption Analysis	Analyse units, cost and usage patterns	Insights into consumption and cost trends
5	Bill Prediction	Predict next month's bill from past data	Estimated bill for planning
6	Graphical Dashboard	Charts and graphs	Easy understanding of usage and expenses
7	Energy Saving Tips	Smart, personalised suggestions	Reduce usage, save money
8	Download Report (PDF)	Generate detailed analysis report	PDF for records

Supporting modules from the blueprint: **Bill Calculator, Appliance-wise Cost Estimator, Alerts, Tariff Configuration.**

4. Functional Requirements

Priority: **P0** = must have (MVP), **P1** = should have, **P2** = nice to have.

4.1 Module 1 — User Registration & Login

ID	Requirement	Priority
FR-1.1	User can register with name, email, password (and optional consumer number).	P0
FR-1.2	Passwords stored hashed (bcrypt); never in plain text.	P0
FR-1.3	Login returns a JWT (or secure session cookie); protected routes require it.	P0
FR-1.4	Logout and token expiry (e.g., 24 h).	P0
FR-1.5	Basic validation (unique email, password $\geq$ 8 chars).	P0
FR-1.6	Profile page: edit name, state/distributor, tariff category (Residential).	P1
FR-1.7	Password reset by email.	P2

4.2 Module 2 — Add / Upload Bill

Two input paths:

A. Manual entry (from blueprint "Inputs")

ID	Requirement	Priority
FR-2.1	Form fields: billing month, previous meter reading, current meter reading, (optional) fixed charge, electricity duty %, other charges, rebates.	P0

ID	Requirement	Priority
FR-2.2	Units consumed = current reading – previous reading. Validate current $\geq$ previous. Example: 1400 – 1250 = 150 units .	P0
FR-2.3	Tariff slabs, fixed charge and duty are pre-filled from the active tariff profile and can be overridden.	P0
FR-2.4	Auto-fill previous reading from the last saved bill.	P1
FR-2.5	Prevent duplicate bills for the same month (warn and offer to update).	P0

B. Upload (from flow chart)

ID	Requirement	Priority
FR-2.6	Accept PDF, JPG, PNG (max 5 MB).	P0
FR-2.7	Backend extracts text: pdfplumber for text PDFs, pytesseract (Tesseract OCR) for images/scanned PDFs.	P1
FR-2.8	Parse key fields with regex/keyword rules: billing month, previous/current reading, units, energy charge, fixed charge, duty, total amount.	P1
FR-2.9	Show extracted values in an editable confirmation screen before saving (OCR is never trusted blindly).	P1
FR-2.10	Store the original file (GridFS) linked to the bill record.	P1

4.3 Bill Calculator (core engine)

ID	Requirement	Priority
FR-3.1	Slab-wise energy charge calculation using configurable slabs.	P0
FR-3.2	Estimated Bill = Energy Charges + Fixed Charges + Taxes/Duty – Rebates (bottom line of blueprint).	P0
FR-3.3	Show a line-by-line breakdown (slab 1, slab 2, fixed, duty, rebate, total).	P0
FR-3.4	"What-if" calculator: enter hypothetical units and see the bill without saving.	P1
FR-3.5	Compare calculated bill with the actual bill amount entered; show variance.	P1

Worked example (illustrative slabs – configurable, not official tariff):

Slab	Units	Rate (₹/unit)	Charge
0–100	100	5.00	500.00
101–300	50	10.00	500.00
Energy charge	150		1000.00
Fixed charge			120.00
Electricity duty (16% of energy + fixed)			179.20
Rebate			0.00
Total			₹1,299.20

*Real tariffs differ by distributor (e.g., MSEDCL in Maharashtra) and change periodically. The app must **not hard-code** rates; admins/users maintain them in the tariff profile.*

4.4 Module 3 – Monthly Bill History

ID	Requirement	Priority
FR-4.1	List all bills in a table: month, units, amount, status.	P0
FR-4.2	View bill detail with full breakdown and attached file.	P0
FR-4.3	Edit and delete a bill (with confirmation).	P0
FR-4.4	Filter by year / date range; sort by amount or units.	P1
FR-4.5	Export history as CSV.	P2

4.5 Module 4 — Consumption Analysis

ID	Requirement	Priority
FR-5.1	Compute: average monthly units, average bill, highest and lowest month, cost per unit (total ÷ units).	P0
FR-5.2	Month-over-month change (% and absolute) in units and cost.	P0
FR-5.3	Seasonal insights (e.g., summer vs. winter usage).	P1
FR-5.4	Identify which slab the user usually lands in and how many units to the next slab.	P1

4.6 Appliance-wise Cost Estimator

ID	Requirement	Priority
FR-6.1	User adds appliances: name, wattage, hours/day, days/month, quantity.	P0
FR-6.2	Monthly kWh per appliance = (Watts × Hours/day × Days × Qty) ÷ 1000.	P0
FR-6.3	Estimated cost per appliance using the user's marginal slab rate (or average rate).	P0
FR-6.4	Rank appliances by consumption; flag top consumers ("high-consumption appliances").	P0
FR-6.5	Preset catalog (AC, fridge, fan, LED bulb, geyser, washing machine, TV, etc.) with typical wattages.	P1
FR-6.6	Show estimated vs. actual total to highlight unaccounted usage.	P2

4.7 Module 5 — Bill Prediction

ID	Requirement	Priority
FR-7.1	Predict next month's units and bill from history.	P0
FR-7.2	v1 method: Linear Regression (scikit-learn) on the last N months; fallback to a 3-month moving average when < 4 data points.	P0
FR-7.3	Display prediction with a confidence range (e.g., ±10%) and the method used.	P1
FR-7.4	Predicted units are passed through the bill calculator so the predicted amount respects slabs.	P0
FR-7.5	Minimum-data message: "Add at least 3 months of bills for prediction."	P0

4.8 Module 6 — Graphical Dashboard (Chart.js)

ID	Chart	Data	Priority
FR-8.1	Line chart	Monthly units trend	P0
FR-8.2	Bar chart	Monthly bill amount / month comparison	P0
FR-8.3	Pie/Doughnut	Appliance-wise share of consumption	P0
FR-8.4	Stacked bar	Bill composition (energy, fixed, duty)	P1
FR-8.5	Line with forecast point	Actual vs. predicted	P1
FR-8.6	KPI cards	Latest bill, units, avg cost/unit, predicted next bill	P0
FR-8.7	Year/month filter	Dashboard-wide	P1

4.9 Alerts

ID	Requirement	Priority
FR-9.1	In-app alert when units exceed the user's monthly threshold or when a month rises > X% vs. previous.	P0
FR-9.2	Alert when usage is about to cross into a higher slab.	P1
FR-9.3	Alert when predicted bill exceeds a user-defined budget.	P1
FR-9.4	Notification bell with read/unread state.	P1
FR-9.5	Email alerts.	P2

4.10 Module 7 — Energy Saving Tips

ID	Requirement	Priority
FR-10.1	Static tip library: use LED bulbs, unplug unused devices, use 5-star rated appliances, optimise AC temperature (24–26 °C), etc.	P0
FR-10.2	Rule-based personalisation: e.g., if AC is the top consumer → AC tips; if usage jumped > 20% → suggest an audit; if close to the next slab → show units to cut.	P0
FR-10.3	Show estimated monthly savings per tip (approximate, e.g., replacing 5 × 60 W bulbs with 9 W LEDs).	P1
FR-10.4	Optional AI-generated tips via the LLM API.	P2

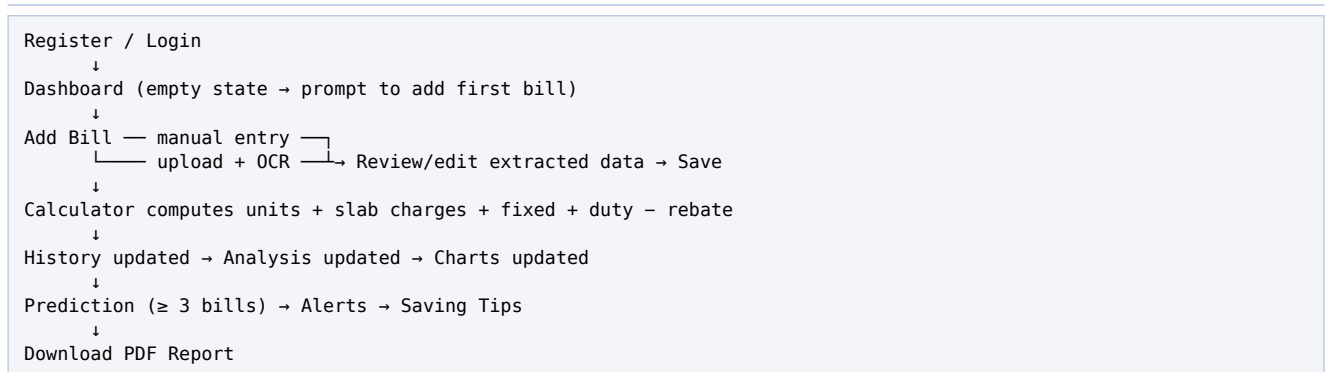
4.11 Module 8 — Download Report (PDF)

ID	Requirement	Priority
FR-11.1	Generate a PDF for a selected month or period using ReportLab (or WeasyPrint).	P0
FR-11.2	Contents: user info, bill breakdown, units, trend charts (rendered via Matplotlib images), appliance ranking, prediction, saving tips.	P0
FR-11.3	Download button on dashboard and bill detail page.	P0

4.12 Tariff Configuration

ID	Requirement	Priority
FR-12.1	Tariff profile: name, distributor, category, slabs (from, to, rate), fixed charge, duty %, other surcharges, effective date.	P0
FR-12.2	User selects an active profile; a default profile is seeded.	P0
FR-12.3	Bills store a snapshot of the tariff used so old bills don't change when tariffs change.	P0

5. User Flow



6. Screens / Pages (HTML)

Page	Key Elements
index.html (Landing)	Product intro, Login/Register CTA
register.html / login.html	Forms with validation and error messages
dashboard.html	KPI cards, line/bar/pie charts, prediction card, alerts, tips
add-bill.html	Tabs: Manual Entry / Upload Bill; live calculation preview
history.html	Bills table, filters, view/edit/delete
bill-detail.html	Breakdown, attached file, download report
appliances.html	Appliance form/list, cost ranking, pie chart
analysis.html	Trends, MoM change, slab insight
tips.html	Personalised and general tips
tariff.html	Manage slabs, fixed charge, duty
profile.html	User settings, thresholds, budget

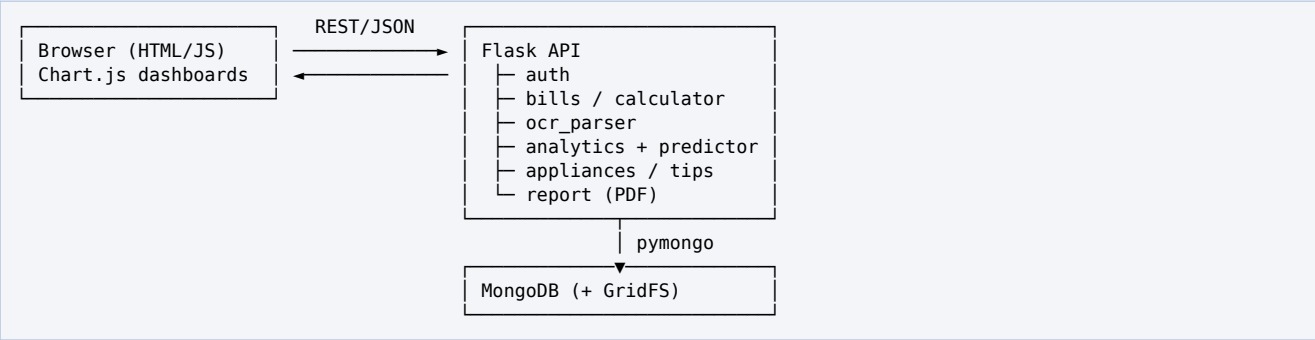
UI guidelines: responsive (mobile-first), blue/yellow palette consistent with the infographics, ₹ currency formatting, clear empty states, accessible contrast and labels.

7. Technical Architecture

7.1 Tech Stack

Layer	Technology
Frontend	HTML5, CSS3 (optionally Bootstrap/Tailwind), Vanilla JavaScript (ES6, fetch), Chart.js
Backend	Python 3.10+, Flask, Flask-CORS, Flask-JWT-Extended
Database	MongoDB (local or Atlas) via pymongo (or Flask-PyMongo/MongoEngine)
File storage	MongoDB GridFS (or local /uploads for MVP)
OCR / Parsing	pdfplumber , pytesseract + Tesseract, Pillow , regex
Prediction	scikit-learn (LinearRegression), numpy , pandas
PDF generation	reportlab (+ matplotlib for chart images)
Security	bcrypt , JWT, python-dotenv
Testing	pytest , Postman
Dev tools	Git/GitHub, VS Code

7.2 High-Level Architecture



7.3 Suggested Folder Structure

```
electricity-bill-analyzer/
├── backend/
│   ├── app.py
│   ├── config.py
│   ├── requirements.txt
│   ├── .env
│   ├── routes/          (auth.py, bills.py, appliances.py, analytics.py, tariff.py, alerts.py,
│   │   │   reports.py)
│   ├── services/        (calculator.py, ocr_parser.py, predictor.py, tips_engine.py, pdf_report.py)
│   ├── models/          (user.py, bill.py, appliance.py, tariff.py, alert.py)
│   ├── utils/           (auth_utils.py, validators.py)
│   └── tests/
├── frontend/
│   ├── *.html
│   ├── css/style.css
│   └── js/               (api.js, auth.js, dashboard.js, addBill.js, history.js, appliances.js, charts.js)
└── README.md
```

8. Data Model (MongoDB Collections)

8.1 users

```
{
  "_id": "ObjectId",
  "name": "Ravi Patil",
  "email": "ravi@example.com",
  "password_hash": "bcrypt-hash",
  "consumer_number": "optional",
  "active_tariff_id": "ObjectId",
  "settings": { "monthly_unit_threshold": 200, "monthly_budget": 1500, "alert_increase_pct": 20 },
  "created_at": "ISODate"
}
```

8.2 tariffs

```
{
  "_id": "ObjectId",
  "user_id": "ObjectId | null (null = system default)",
  "name": "Residential Default",
  "distributor": "MSEDCL",
  "slabs": [
    { "from": 0, "to": 100, "rate": 5.0 },
    { "from": 101, "to": 300, "rate": 10.0 },
    { "from": 301, "to": null, "rate": 14.0 }
  ],
  "fixed_charge": 120,
  "duty_percent": 16,
  "other_charges": 0,
  "effective_from": "ISODate"
}
```

8.3 bills

```
{
  "_id": "ObjectId",
  "user_id": "ObjectId",
  "month": "2026-05",
  "previous_reading": 1250,
  "current_reading": 1400,
  "units": 150,
  "breakdown": {
    "slab_charges": [
      { "range": "0-100", "units": 100, "rate": 5, "amount": 500 },
      { "range": "101-300", "units": 50, "rate": 10, "amount": 500 }
    ],
    "energy_charge": 1000,
    "fixed_charge": 120,
    "duty": 179.2,
    "other_charges": 0,
    "rebate": 0
  },
  "calculated_total": 1299.2,
  "actual_total": 1310.0,
  "tariff_snapshot": { "...": "copy of tariff used" },
  "source": "manual | upload",
  "file_id": "GridFS ObjectId | null",
  "created_at": "ISODate",
  "updated_at": "ISODate"
}
```

Indexes: { user_id: 1, month: -1 } (**unique** compound to block duplicates).

8.4 appliances

```
{
  "_id": "ObjectId", "user_id": "ObjectId",
  "name": "Air Conditioner", "watts": 1500,
  "hours_per_day": 6, "days_per_month": 30, "quantity": 1,
  "monthly_kwh": 270, "estimated_cost": 2400
}
```

8.5 alerts

```
{
  "_id": "ObjectId", "user_id": "ObjectId",
  "type": "high_usage | slab_warning | budget | spike",
  "message": "Usage is 28% higher than last month.",
  "is_read": false, "created_at": "ISODate"
}
```

8.6 predictions (optional cache)

```
{ "user_id": "ObjectId", "for_month": "2026-06", "predicted_units": 172, "predicted_amount": 1480,
  "method": "linear_regression", "created_at": "ISODate" }
```

9. API Specification (REST, JSON)

Base URL: `/api/v1` · Auth: `Authorization: Bearer <JWT>` (except auth routes)

Method	Endpoint	Description
POST	<code>/auth/register</code>	Create account
POST	<code>/auth/login</code>	Login → JWT
GET	<code>/auth/me</code>	Current user profile
PUT	<code>/auth/me</code>	Update profile / settings
POST	<code>/bills</code>	Create bill (manual) — runs calculator, saves, triggers alerts
POST	<code>/bills/upload</code>	Upload PDF/image → returns extracted fields for review
GET	<code>/bills</code>	List bills (filters: year, from, to)

Method	Endpoint	Description
GET	/bills/<id>	Bill detail
PUT	/bills/<id>	Update bill (recalculates)
DELETE	/bills/<id>	Delete bill
POST	/calculate	Stateless calculator (what-if)
GET	/analytics/summary	Averages, min/max, MoM change, cost per unit
GET	/analytics/charts	Datasets for line/bar/stacked charts
GET	/predict	Next-month prediction
GET/POST	/appliances	List / add appliance
PUT/DELETE	/appliances/<id>	Edit / remove appliance
GET	/appliances/summary	Ranking + pie data
GET	/tips	Personalised tips
GET	/alerts	List alerts
PATCH	/alerts/<id>/read	Mark read
GET/POST/PUT	/tariffs	Manage tariff profiles
GET	/reports/pdf?month=YYYY-MM	Download PDF report

Standard response format

```
{ "success": true, "data": { }, "message": "OK" }
{ "success": false, "error": { "code": "VALIDATION_ERROR", "message": "Current reading must be ≥ previous reading" } }
```

10. Key Algorithms

10.1 Slab-wise Calculation

```
def calculate_bill(units, tariff, rebate=0):
    remaining, energy, lines = units, 0, []
    for s in tariff["slabs"]:
        cap = (s["to"] - s["from"] + 1) if s["to"] else float("inf")
        if s["from"] == 0:
            cap = s["to"] # first slab: 0..to
        used = min(remaining, cap)
        if used <= 0:
            break
        amt = used * s["rate"]
        lines.append({"units": used, "rate": s["rate"], "amount": amt})
        energy += amt
        remaining -= used
    fixed = tariff["fixed_charge"]
    duty = (energy + fixed) * tariff["duty_percent"] / 100
    total = energy + fixed + duty + tariff.get("other_charges", 0) - rebate
    return {"lines": lines, "energy": energy, "fixed": fixed, "duty": duty, "total": round(total, 2)}
```

10.2 Prediction

1. Fetch the last up-to-12 bills sorted by month.
2. If $n < 3$ → return "insufficient data".
3. If $3 \leq n < 6$ → 3-month moving average.
4. If $n \geq 6$ → [LinearRegression](#) on (month_index → units); optionally add month-of-year seasonality feature.
5. Predicted units → [calculate_bill\(\)](#) → predicted amount.

10.3 Bill Field Extraction (OCR)

1. Detect file type → PDF text (pdfplumber) or image OCR (pytesseract, grayscale + threshold pre-processing).

2. Normalise text; apply regex patterns for keywords such as *Previous Reading*, *Current Reading*, *Units*, *Fixed Charge*, *Duty*, *Total/Net Payable*.
3. Return fields with a confidence flag; UI highlights low-confidence values for manual correction.

10.4 Rule-based Tips Engine

Rule	Tip
Top appliance is AC	Set 24–26 °C, clean filters, use inverter/5-star AC
Lighting share > 15%	Replace with LED bulbs
Units ↑ > 20% vs. last month	Check for standby loads / new appliance
Within 15 units of the next slab	"Cut ~X units to stay in the cheaper slab"
Old fridge/geyser flagged	Consider 5-star rated replacement

11. Non-Functional Requirements

Category	Requirement
Performance	API p95 < 500 ms (excluding OCR/PDF); OCR < 10 s per file
Security	Hashed passwords, JWT auth, input validation/sanitisation, CORS whitelist, file type/size checks, rate limiting on auth routes, HTTPS in production, secrets in <code>.env</code>
Privacy	Bills are personal data; each user can only access their own records; delete-account removes all data
Reliability	Graceful error messages; DB indexes; validation on both client and server
Usability	Responsive on mobile/desktop; ≤ 3 clicks to add a bill; clear empty states
Compatibility	Latest Chrome, Edge, Firefox, Safari
Maintainability	Modular services, PEP-8, docstrings, README with setup steps
Scalability	Stateless API; MongoDB indexes; can be containerised (Docker) later

12. Testing Plan

Type	What	Tools
Unit	Calculator (low/medium/high units, slab boundaries 100/101/300/301, zero units), appliance kWh formula, prediction fallback logic	pytest
Accuracy	Compare calculated bill with an actual bill from the user's distributor; variance ≤ 2%	Manual
Boundary	Low / medium / high unit scenarios (e.g., 50, 150, 450 units)	pytest
API	Auth, CRUD, validation errors, unauthorised access	Postman / pytest
OCR	5–10 sample bills (PDF and photo) — field accuracy	Manual
UI	Form validation, chart rendering, responsive layout	Manual checklist
Security	Access another user's bill (must fail), oversized/invalid file upload, injection strings	Manual
Report	PDF opens correctly and matches dashboard values	Manual

Acceptance criteria (MVP): a user can register, add a bill (150 units test case yields the expected breakdown), see it in history and charts, get a prediction after 3+ bills, view appliance ranking and tips, and download a PDF.

13. Milestones (Suggested 6-Week Plan)

Week	Deliverables
1	Requirements, DB schema, project setup, Flask skeleton, MongoDB connection, tariff seed
2	Auth (register/login/JWT), landing/login/register pages
3	Bill calculator, manual bill entry, history (CRUD), analysis summary
4	Dashboard with Chart.js, appliance module, alerts
5	Prediction, tips engine, upload + OCR extraction with review screen
6	PDF report, testing, bug fixes, documentation, demo

14. Risks & Mitigations

Risk	Impact	Mitigation
OCR inaccurate on varied bill layouts	Wrong data	Mandatory review screen; manual entry always available; start with one distributor's format
Tariffs differ / change	Wrong calculation	Configurable tariff profiles with snapshots; user-editable
Too little data for prediction	Poor forecast	Fallback methods, minimum-data message, show range
Scope creep	Delay	Strict P0 → P1 → P2 order; OCR and IoT are optional
Tesseract setup issues	Blocks upload	Document install; fallback to manual entry

15. Future Scope

1. **Smart meter / IoT integration** for live consumption (MQTT, ESP32 + current sensor).
2. **Mobile app** (PWA first, then React Native/Flutter).
3. **Advanced prediction** (ARIMA/Prophet/LSTM with weather data).
4. **Solar analysis** — payback period, net-metering savings estimate.
5. Multi-user households with bill splitting.
6. Auto-fetch bills from distributor portals / email.
7. WhatsApp/SMS/email alerts and bill-due reminders.
8. Carbon footprint tracking.
9. Admin panel and multi-language support (English, Marathi, Hindi).

16. Assumptions & Open Questions

- Project is for **residential** consumers first.
- Currency ₹ (INR), units in kWh.
- **Open:** Which distributor's tariff and bill layout will be the reference for testing and OCR (e.g., MSSEDCL)?
- **Open:** Is OCR upload mandatory for submission, or is manual entry sufficient for the mini project?
- **Open:** Will the app be deployed (Render/Railway + MongoDB Atlas) or shown locally?

17. Glossary

Term	Meaning
Slab	Unit range billed at a specific per-unit rate
Fixed charge	Monthly charge independent of consumption
Electricity duty	Government tax, usually a % of charges
kWh / Unit	1 unit = 1 kilowatt-hour
MoM	Month over month
OCR	Optical Character Recognition
JWT	JSON Web Token